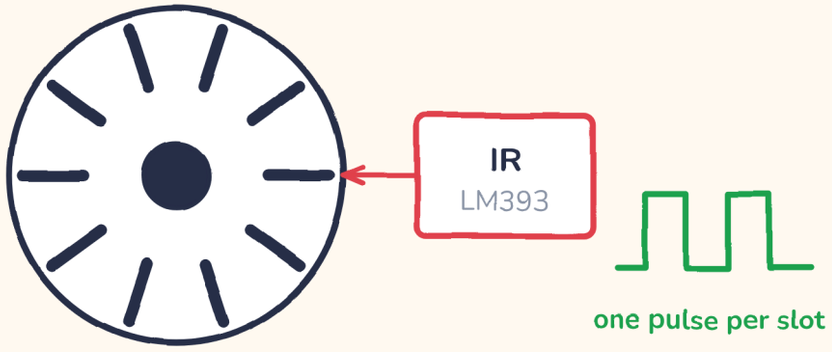
Robotics · Motors & Motion Control

How Far Did I Go?

Robots can't see their own wheels. So how do they know they drove one metre?



20 slots = 20 ticks per wheel turn

90 minutes | Pairs | Real kit

Did you know?

Ancient Roman carts carried an odometer: a gear that dropped one pebble into a box every Roman mile. Counting wheel turns is a two thousand year old idea.

Let's think

Why 'drive for 2 seconds' always fails

The same code, four different answers

Run `forward(); delay(2000);` four times and measure.

- Fresh batteries: 118 cm
- Half-flat batteries: 94 cm
- On carpet: 86 cm
- Carrying a book: 71 cm

Time is not distance

Time only becomes distance if speed never changes. On a real robot, speed changes constantly.

So measure the thing you care about

Do not guess from time. **Count the wheel.**

If you know how far the wheel travels in one turn, and you can count turns, you know distance. Battery level, carpet and cargo stop mattering.

This single idea is what separates a toy from a robot.

Learning aims

Today I will learn to…

1. Explain **encoder ticks** and how wheel circumference turns them into distance
2. Count ticks with an **interrupt**, not a polling loop
3. Drive an **exact distance** and measure the error
4. Work out **speed** from ticks per second

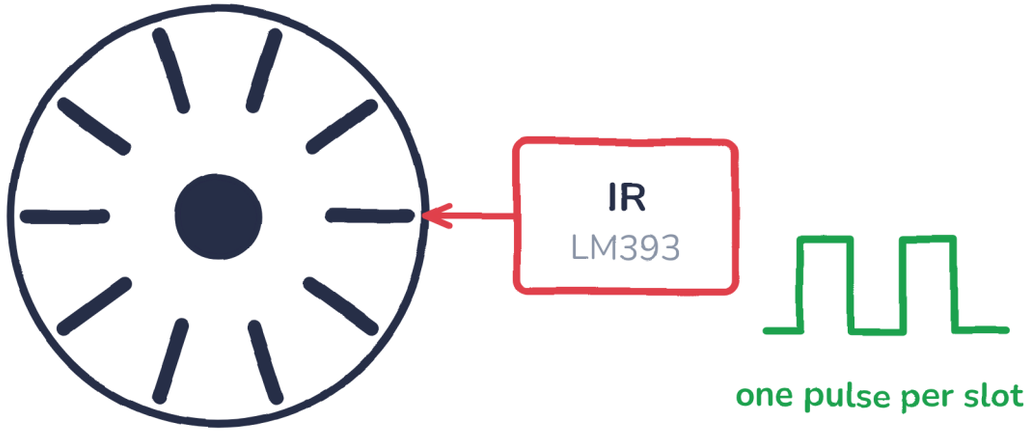
Keywords

encoder · tick · slot · interrupt · ISR · volatile · odometry · circumference · resolution · debounce

Objective 1 · The sensor

A slotted disc and an eye

Light, blocked, light, blocked



20 slots = 20 ticks per wheel turn

How it works

An infrared LED shines at a phototransistor. The slotted disc on the axle chops the beam. The LM393 turns each chop into a clean HIGH-LOW edge.

Resolution

20 slots means 20 ticks per turn. That is the finest distance you can ever measure: about **1 cm** on a 65 mm wheel.

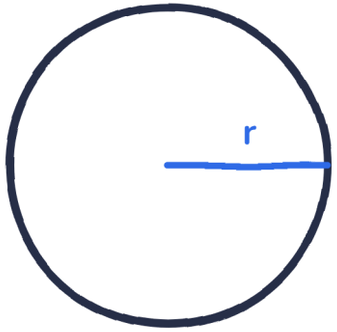
One beam cannot tell direction

Forward and backward look identical. Two sensors, slightly offset, would solve it. That is a quadrature encoder.

Objective 1 · The maths

From ticks to centimetres

Do this once, with your own wheel



wheel

circumference = π × diameter

65 mm wheel → 204 mm per turn

20 ticks per turn → 10.2 mm per tick

1 metre = 98 ticks

measure your own wheel. Never trust the box.

Work it through

Circumference = π × d = 3.1416 × 65 mm = **204 mm**.

204 ÷ 20 slots = **10.2 mm per tick**.

1000 ÷ 10.2 = **98 ticks** for one metre.

Measure, do not assume

A "65 mm" wheel with a rubber tyre is often 66 or 67 mm. Roll it along a ruler for one turn and use *that* number.

LEARN

Objective 2 · Interrupts

Why loop() is not fast enough

The problem with polling

At full speed a wheel may produce **200 ticks a second**. If your loop also prints to Serial and reads a sensor, it may only come round 50 times a second.

Three quarters of your ticks are simply never seen, and your distance is quietly wrong.

Missed ticks never come back

There is no way to notice afterwards. The count is just low, and everything built on it is low too.

What an interrupt does

You tell the chip: "whenever pin 2 goes from LOW to HIGH, drop everything and run this function."

The main loop is paused mid-instruction, the counter is bumped, and the loop resumes. Nothing is missed.

Two rules for an ISR

Keep it **tiny**: no Serial.print, no delay. And mark any variable it touches **volatile**, or the compiler will optimise your reads away.

In the real world

Counting is everywhere

Your bike computer

A magnet on a spoke and a reed switch. One pulse per wheel turn, and it knows your speed and your distance.

A computer mouse

Old ball mice had two slotted wheels inside, one for each axis, doing exactly what your robot is about to do.

A lift

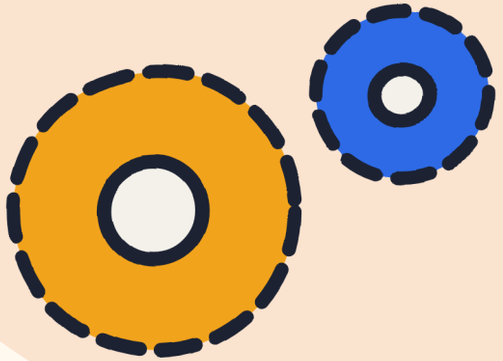
An encoder on the motor tells the controller which floor it is passing, so it knows when to start slowing down.

Part 2 · Make it

Now let's count

Fit the sensor, count ticks, then drive exactly one metre and measure the error.

50 minutes | Pairs | Real kit



Class game

Count the wheel by hand

How to play

1. Mark one spot on a wheel with tape.
2. Push the robot one metre along a measured line.
3. Count the turns out loud, including the fraction.
4. Now work backwards: how far is one turn?

You have just performed odometry with your eyes. The encoder is only doing it faster.

Round two: slip

Push the robot but hold one wheel so it slides. Ticks say you moved; the floor says you did not. **Encoders measure the wheel, not the world.**

Remember this

Wheel slip is why a robot that only counts ticks will still get lost eventually.

MAKE

Build step 1 of 3

Fit the sensor and see a tick

Do this

1. Slide the slotted disc onto the axle, inside the chassis.
2. Clamp the LM393 so the disc runs through its gap.
3. VCC to 5V, GND to GND, **OUT to pin 2**.
4. Turn the wheel by hand. The LED on the module should flicker.

LED stays on or off?

The disc is not in the gap, or the little trimmer needs adjusting. Turn it until the LED just flickers as the disc moves.

```
// see the raw signal first
void setup() {
  pinMode(2, INPUT);
  Serial.begin(9600);
}

void loop() {
  Serial.println(digitalRead(2));
  delay(20);
}

// Turn the wheel slowly by hand.
// You should see a clean run of
// 1 1 1 0 0 0 1 1 1 0 0 0
// If it is noisy, the sensor is
// not aligned with the slots.
```

MAKE

Build step 2 of 3

Count with an interrupt

Do this

1. Attach the interrupt to pin 2, RISING edge.
2. Spin the wheel one full turn by hand.
3. Does it print 20? If not, your disc has a different slot count.
4. Use *your* number everywhere from now on.

Why the noInterrupts block?

The count is two bytes. An interrupt landing between reading byte one and byte two would hand you nonsense.

```
volatile long ticks = 0;

void onTick() { ticks++; }     // keep it tiny

void setup() {
  pinMode(2, INPUT);
  attachInterrupt(digitalPinToInterrupt(2),
                  onTick, RISING);
  Serial.begin(9600);
}

void loop() {
  noInterrupts();
  long t = ticks;              // safe copy
  interrupts();

  Serial.print("ticks: ");
  Serial.println(t);
  delay(250);
}
```

MAKE

Build step 3 of 3

Drive exactly one metre

Do this

1. Mark a 1 m line on the floor with tape.
2. Run the sketch. Measure where it actually stopped.
3. Write the error down. Repeat five times.
4. Now do it again with half-flat batteries. Did the error change?

The point of the whole lesson

With time-based driving, flat batteries ruin the distance. With ticks, they do not. Prove it to yourself.

```
const float MM_PER_TICK = 10.2;

void driveMM(int mm) {
  noInterrupts(); ticks = 0; interrupts();
  long target = mm / MM_PER_TICK;

  forward();
  while (true) {
    noInterrupts();
    long t = ticks;
    interrupts();
    if (t >= target) break;
  }
  halt();

  Serial.print("stopped after ");
  Serial.println(ticks);
}

void loop() { driveMM(1000); delay(4000); }
```

Test like an engineer

Where the error comes from

Try this	What happens	What it teaches
Drive on tile, then on carpet	Carpet is short by a few cm	Wheel slip: ticks are not ground truth
Run at speed 255 instead of 150	It overshoots further	The robot cannot stop instantly
Use flat batteries	Distance stays right, time changes	This is exactly why we count
Cover the sensor with a finger	Count freezes, robot never stops	Always add a timeout as a safety net
Count 10 turns, divide by 10	A much better mm-per-tick	Averaging beats one measurement

Overshoot

Stopping at exactly the target tick still overshoots, because the robot has momentum. Subtract a few ticks, or slow down near the end.

LEVEL UP

Finished early?

Go further

1 

Work out **speed**: ticks in a 100 ms window × 10

Print it live and push the robot by hand.

2 

Fit **two encoders** and keep both wheels at the same count

This is how a robot drives genuinely straight.

3 

Turn exactly **90 degrees** using ticks instead of time

Work out the arc length from the wheel spacing.

4 

Add a **slow-down zone** in the last 50 mm

Overshoot drops to almost nothing.

Check yourself

Can I really do it? Prove it.

1 Explain **ticks and circumference**

Proof: I worked out my own mm per tick from a measured wheel

2 Count ticks with an **interrupt**

Proof: one hand turn printed exactly my slot count

3 Drive an **exact distance**

Proof: within 3 cm of a metre, five times running

4 Work out **speed** from ticks per second

Proof: the number rose when I pushed the robot faster

Rate yourself: red = not yet · amber = nearly · green = yes, and I can show someone

Show what you know

What did we learn?

Five things to keep

- Time is not distance. Count the wheel.
- Circumference ÷ slots = millimetres per tick.
- Interrupts catch ticks that loop() would miss.
- ISRs stay tiny; shared variables are volatile.
- Encoders measure the wheel, not the world. Slip is real.

Exit ticket

Your robot reports 1.00 m but the tape says 0.92 m. Give two possible causes and how you would tell them apart.

Next class

ROB-08: feedback control. Now that the robot can measure itself, it can correct itself.